

Data Structures 1 – Winter 2025

Assignment A3

General Rules

Item	Details
Deadline	December 27, 2025, 11:59 p.m.
Late Penalty	10 pts if submitted Dec 28, 20 pts if submitted Dec 29. No submissions accepted after Dec 30
Group Size	Pairs (two students)
Oral Defense	You may be asked to explain your solution orally. Failure to answer satisfactorily implies a grade 0 in each exercise involved in the discussion. Dividing the task is not excuse. You must be able to explain every exercise.
Academic Integrity	Any detected plagiarism or copying will be reported to the UG office. Copy/Paste from IA is plagiarism.
Language & Environment	Java (must compile in VS Code and pass all provided tests).
Deliverables	One zip file containing all Java source files (<i>wet part</i>). One PDF file with all written answers (<i>dry part</i>). We won't accept hand written assignments. The dry part must be typewritten in computer.

1. [15 pts] Sorting algorithms.

- a. [8 pts] Given the quicksort algorithm using the Hoare and Lomuto partitions as presented in the lectures.
1. Provide one example for which Lomuto partition is faster than Hoare partition.
 2. Provide one example for which Hoare partition is faster than Lomuto partition.
 3. Provide a general argument on when one partition approach is better than the other.
- b. [2 pts] Provide the pseudo code of quicksort using the Hoare partition.
- c. [2 pts] Implement quicksort using the Hoare partition and Lomuto partition in Java (the use of do while, while true, break and continue statements is strictly forbidden).
- d. [3 pts] Provide a program that effectively shows that the runtime of the examples provided in item (a) are really faster when running the implementation. Hint: you may need to execute the sorting many times to collect average times.

2. [10 pts] Hash Tables and Collisions (Dry)

- a. [5 pts] We want to insert keys into a Hash Table of size $m = 11$ using the hash function $h(k) = k \bmod 11$.
- Insert the keys: 12, 25, 45, 15, 1, 23, 95.
 - Show the table content using **Open Addressing with Linear Probing**.
 - Show the table content using **Open Addressing with Double Hashing**, where $h_2(k) = 1 + (k \bmod 10)$.
- b. [5 pts] Prove that in a hash table with chaining (close addressing), under the assumption of simple uniform hashing, the expected time for an unsuccessful search is $O(1 + \alpha)$, where α is the load factor.
-

3. [20 pts] Working with tries (Dry and Wet)

Given a Trie that is designed using using `TrieNode`, nodes that contains an array of 26 child nodes (see tutorial 8).

- [5 pts] Write the pseudocode of a function that takes a Trie and converts it into a Patricia Trie.
- [5 pts] Write the pseudocode of search, insert and remove in a Patricia Trie.
- [7 pts] Implement the pseudocodes of the 2 previous items in Java
- [3 pts] Provide meaningful test.

4. [20 pts] Montecarlo approach for assessing the expected height of Treap and BST

We recall the principles of the Monte Carlo approach, as introduced in Introduction to Computer Science M. The Monte Carlo method is a computational technique that relies on random sampling to solve problems that, although theoretically solvable analytically, may be difficult or complex to resolve directly. The core idea is to simulate many random outcomes of a process, observe the results, and use statistical analysis to estimate the quantity of interest.

- [10 pts] Use the Monte Carlo approach to assess whether the expected height of a Treap with n elements is $O(\log n)$. You may support your conclusion with analytical or graphical representations of your results.
- [5 pts] Use the same set of inputs sequences used to build the Treaps to build Binary Search trees. Compare pairwise the difference of heights between Treap and BST (for the same cases). Provide an analysis of the maximum, minimum, average and median heights.
- [5 pts] Use the same of inputs sequences used to build a Hashtable with close addressing. Compute the load factor for every sequence and compare average execution times of Treap, BST and Hashtable. Provide an analysis of the maximum, minimum, average and median execution times.

Hints: - Modify your Treap, BSTs and Hashtable implementations as needed to collect relevant data. - Write a Java program that uses the Monte Carlo method to estimate the expected height of randomly generated Treaps for various values of n (using random input sequences of size n). Use a value of n big enough (e.g, 1000). - You can create sequences of inputs, of size between $[0..n]$. - Even sequences are random, make sure you use the **same** sequences for the 3 data structures. - Your program does not need to generate plots; it should simply output the collected data. You can analyze this data graphically later using external tools or even manually. - Be sure to include the data used to support your analysis.

5. The Ultimate Blockchain – 35 pts (Wet)

Problem Description Extend the system to create the **Ultimate Blockchain (UB)**. This version introduces some cryptographic integrity for both **transactions** and the **global state** (i.e., balances of addresses).

Core Changes

- **Block Hash:** Blocks hash are no longer integer but a string computed in the following way:
 $Hash_{Block} = Hash(previousHash + stateRootHash + TxHash)$ (more details later).
- **Basic Transaction Integrity:** The integrity of all transactions within a block is maintained by computing an aggregated hash function called `txHash`.
- **State Integrity (Merkle Trie):** Account balances are stored in a **Simplified Merkle Trie (SMT)**. The block header stores the `StateRoot` representing the balances *after* the transactions are executed.

5.1 Updated BalanceImp using Simplified Merkle Trie (SMT) The new `BalanceImp` class uses a **SMT**. The SMT is a Trie that has two purposes:

- Is a dictionary from addresses to balances.
- It maintains the consistency of the trie by computing the hash of its children.

Assumption: Addresses are strings of digits 0-9 (e.g., “105”, “404”) and they have at most 10 digits.

Every node n is computed out of the hash of its children plus (the string representation of) its balance, in case of a leaf or end-word, or the string “null” for nodes without balances.

- $hash_n = hash(balance_or_null + \sum_{c \in child(n)} hash_c)$.

Where *balance_or_null* is the string representation of the balance if it is not null or “null” otherwise. Since the Trie will be implemented using arrays for the digits, in case $child[d] = null$, we define $hash_d = “0”$.

When a balance changes, the **hash** must be recalculated. The hash change must propagate **upwards** to the root.

Operations:

- **update(String address, int balance)**: Updates balance and recalculates hashes up to root in the SMT. In the address doesn’t exist it insert the address in the SMT.
 - Complexity: $O(L)$ where L is address length.
- **get(String address)**: Returns balance. $O(L)$.
- **getRootHash()**: Returns the hash of the root node. $O(1)$.
- **repOK()**: As un previous balance implementations, the sum of the balance is equal to the **totalSupply**. In this case, the representation invariant for SMT must hold. That is, SMT is a trie where hashes of parent nodes is the sum of hashes of their children.

The class **BalanceImp** must provide this operations and will be implemented using the SMT (the Simple Merkle Trie).

5.2 Blockchain Integration Class Block Updates:

- **setStateRootHash(String stateRootHash)**: update the stateRootHash for the Block. The stateRootHash comes from the current state of the balance. $O(1)$.
- **getStateRootHash()**: get the stateRootHash for the block. $O(1)$
- **computeAndSetBlockHash()**: computes and set the hash of the block using this formula:
 - $hash_{Block} = Hash(previousHash + stateRootHash + TxHash)$. $O(1)$.
- **computeTxHash()**: computes the for the transactions in the block. Returns the computed **TxHash**.
 - Assumes the block is full.
 - **TxHash** is the $hash(Th_n \dots + hash(Th_3 + hash(Th_2 + hash(Th_1 + hash(Th_0))))))$ where T_i is the result $T_i.hash()$ for the transaction T_i
 - Complexity $O(|T|)$

Class UBlockChain Updates:

- **processCurrentBlock()**: it has the same functionality as the previous assignment but now also:
 - Computes **txHash** with all transactions to be processed.
 - Execute transactions and update balances using the **StateSMT** for balances.
 - Updates the **StateSMT** accordingly (triggering hash updates).
 - Retrieve the new **stateRoot** from the **StateMPT**.
 - Save the new **stateRoot** for the Block.
 - Computes the $Hash_{Block}$ using the computed **stateRoot** and **txJHash**.

Implement the function in the most efficient way you can and provide the new bound of complexity.

- **repOK()**: update it to validate consistency of the blockchain
 - The stateRoot must be consistent with the balance on each block
 - that **currentBlock.previousHash == previousBlock.blockHash**.
 - Verify for the last inserted block that **blockHash** is valid based on its contents.
 - Complexity: provide the new bound of complexity.

Implementation Notes

- Use your own structures. You cannot use Java built-in classes.

- You must implement the code using the code template. In the code template there is a basic solution to the assignment 2 that you can use as a baseline for assignment 3. You will see the class `UBlockchain` that extends the class `ABlockchain`. So, in principle you need to only implement and overwrite a few methods in the new class.
- Use a helper function `HashUtils.hash(String)` included in the code template.
- Use the function `Transaction.hash()` to get the hash of the transaction.
- The solution need to pass the provided tests.
- Add new test of the solutions.
- Use of IA, and other forms of plagiarism are strictly forbidden.